

SDD con GitHub Spec Kit

Sistema de Biblioteca

Partimos de un enunciado de biblioteca con alta, baja y modificación de libros, clientes y préstamos. También incluía requisitos no funcionales: usabilidad, performance, escalabilidad, reglas legales y calidad del diseño.

Idea central

SDD no es pedirle a la IA que programe. Es obligarla a producir decisiones verificables antes de escribir código.

Integrar GitHub Specs Kit a un repo existente

```
uvx --from git+https://github.com/github/spec-kit.git specify init .
```

Repositorios

<https://github.com/MartinBarbieri/biblioteca-sdd>

<https://github.com/MartinBarbieri/biblioteca-backend>

Flujo completo de trabajo

Paso	Comando / acción	Pregunta docente	Artefacto generado	Qué mirar
1	/speckit.constitution	¿Qué reglas no se negocian?	constitution.md	Principios: spec-first, POO, capas, tests.
2	/speckit.specify	¿Qué debe hacer el sistema?	spec.md	US, FR, NFR, edge cases, ambigüedades.
3	/speckit.clarify	¿Qué no está claro?	spec.md actualizado	Decisiones sobre roles, disponibilidad, ejemplares.
4	/speckit.checklist	¿La spec es testeable?	functional-quality.md	Complejidad, claridad, consistencia.
5	Corrección guiada	¿Qué gaps hay antes de planificar?	spec.md corregido	FR duplicados, performance, responsabilidades.
6	/speckit.plan	¿Cómo se construye?	plan, research, data-model, openapi, quickstart	Stack, arquitectura, modelo, API.
7	/speckit.tasks	¿Qué tareas exactas hay?	tasks.md	Tareas con ID, dependencia y archivos.
8	/speckit.analyze	¿Todo traza con todo?	reporte + correcciones	Requisitos sin tarea, tareas sin requisito.
9	taskstoissues / gh CLI	¿Cómo se gestiona el trabajo?	GitHub Issues	Un issue por tarea, labels y trazabilidad.
10	Implementar T001	¿Cuál es el mínimo código controlado?	pom.xml + clase principal	Solo una tarea; no avanzar con otra.

Prompts completos

Constitution

/speckit.constitution

Este proyecto es un sistema de biblioteca usado con fines educativos para enseñar programación orientada a objetos, SDD y desarrollo full-stack.

Principios obligatorios:

1. La especificación manda sobre la implementación. No se debe programar funcionalidad que no esté trazada a una historia de usuario o requisito.
2. El dominio debe modelarse con orientación a objetos real. Evitar clases anémicas cuando haya reglas de negocio claras.
3. Separar responsabilidades:
 - dominio: reglas de negocio
 - aplicación/servicios: casos de uso
 - infraestructura: base de datos, repositorios, persistencia
 - presentación/API: endpoints y DTOs
 - frontend: interfaz de usuario
4. En el backend Java se debe evitar lógica duplicada, métodos gigantes, validaciones mezcladas con controladores y uso innecesario de instanceof.
5. Usar enums o value objects cuando representen valores cerrados, por ejemplo categoría de libro.
6. Aplicar patrones de diseño solo si simplifican el diseño. No forzar patrones.
7. Toda regla de negocio importante debe tener tests.
8. El sistema debe poder crecer a 100.000 libros y 10.000 clientes, por lo que las búsquedas de disponibilidad no deben depender de recorrer listas completas en memoria.
9. La interfaz debe priorizar usabilidad: encontrar libros disponibles debe ser simple, rápido y claro.
10. No implementar código durante las fases de especificación, aclaración, checklist, planificación y generación de tareas. Solo crear o actualizar los documentos correspondientes.

Specify

/speckit.specify

Desarrollar un sistema de biblioteca que permita gestionar libros, clientes y préstamos. Objetivo:

La biblioteca necesita administrar el alta, baja y modificación de libros, clientes y préstamos. Los clientes deben poder encontrar libros disponibles de forma simple y rápida, y el sistema debe permitir registrar préstamos y devoluciones.

Actores principales:

- Bibliotecario: administra libros, clientes y préstamos.
- Cliente: consulta libros disponibles y puede tomar varios libros prestados.
- Sistema: valida disponibilidad, registra operaciones, genera informes y asigna fotos de perfil.

Requisitos funcionales:

1. Gestión de libros:
 - Registrar un nuevo libro.
 - Modificar los datos de un libro existente.
 - Dar de baja un libro.
- Para cada libro se debe cargar:
 - título
 - autor
 - cantidad de páginas

- editorial
- ISBN
- si pertenece a una saga o no
- categoría: infantiles, juveniles, adultos o conocimiento
- estado de copyright vigente

2. Gestión de clientes:

- Registrar un nuevo cliente.
- Modificar datos de un cliente.
- Dar de baja un cliente.
- Para cada cliente se debe almacenar:
 - nombre
 - apellido
 - DNI
 - fecha de nacimiento

- foto de perfil asignada automáticamente
- puntos acumulados

3. Foto de perfil:

- Cada vez que se registra un cliente nuevo, el sistema debe asignarle como foto de perfil una foto de un animal.
- El animal debe empezar con la misma inicial que el nombre de pila del cliente.

4. Préstamos:

- Un cliente puede tomar prestados varios libros.
- Antes de prestar un libro, el sistema debe consultar si el libro está disponible.
- Si el libro está disponible, el sistema debe registrar el préstamo.
- El préstamo debe tener fecha de inicio y fecha final prevista.
- Cuando un cliente devuelve un libro antes de la fecha final del préstamo, se le deben sumar 10 puntos por entrega anticipada.

5. Disponibilidad:

- El sistema debe permitir consultar la disponibilidad de un libro.
- La consulta de disponibilidad no debe tardar más de 2 segundos.
- La disponibilidad debe considerar los préstamos activos.

6. Informes:

- El sistema debe generar un informe de libros prestados en cualquier momento.
- El informe debe mostrar los libros actualmente prestados y a qué cliente corresponde cada préstamo.

Requisitos no funcionales:

1. Usabilidad:

- La interfaz debe ser amigable.
- Debe ser fácil encontrar libros disponibles.
- La información de disponibilidad debe verse de manera clara.

2. Performance:

- La consulta de disponibilidad de un libro no debe exceder los 2 segundos.
- El sistema debe poder manejar al menos 100.000 libros y 10.000 clientes.

3. Reglas legales:

- Todos los libros registrados deben tener el copyright al día según los requisitos legales del país.

4. Calidad:

- Las reglas de negocio deben estar separadas de la interfaz.
- Las validaciones importantes deben estar explicitadas como criterios de aceptación.

Fuera de alcance inicial:

- Pagos.
- Multas económicas.
- Compra de libros.
- Recomendaciones automáticas.
- Login avanzado por roles, salvo que se considere necesario para separar bibliotecario y cliente.

Generar una especificación con:

- historias de usuario
- prioridades
- criterios de aceptación
- requisitos funcionales numerados
- requisitos no funcionales

- casos borde
- ambigüedades marcadas explícitamente

No implementar código.

Plan técnico

/speckit.plan

Planificar la implementación full-stack del sistema de biblioteca definido en:

specs/002-library-management/spec.md

No implementar código todavía.

No crear frontend ni backend todavía si no es necesario.

Solo generar o actualizar los documentos técnicos de planificación correspondientes.

Contexto del proyecto:

Este repo es el repo meta de especificación end-to-end. El proyecto final tendrá:

- backend/
- frontend/

Backend:

- Java 21
- Spring Boot 3
- Maven
- Spring Web
- Spring Data JPA
- Bean Validation
- PostgreSQL
- Flyway para migraciones
- JUnit 5 y Mockito para tests
- OpenAPI para documentar contratos REST

Frontend:

- React con Vite
- TypeScript
- CSS simple
- Cliente HTTP para consumir la API REST
- Pantallas separadas para libros, clientes, préstamos, devoluciones, disponibilidad e informes

Arquitectura backend:

- Separar domain, application, infrastructure y api.
- Los controladores no deben contener reglas de negocio.
- Los servicios de aplicación coordinan casos de uso.
- Las entidades de dominio contienen reglas propias cuando corresponda.
- Los repositorios abstraen la persistencia.
- Evitar lógica duplicada y evitar uso innecesario de instanceof.

Modelo de dominio esperado:

- Ejemplar
- Cliente
- Prestamo
- FotoPerfilAnimal
- SelectorFotoPerfilAnimal
- CategoriaLibro como enum
- EstadoEjemplar como enum
- EstadoPrestamo como enum

Reglas importantes:

- Un ejemplar está disponible solo si está activo, tiene copyright vigente y no tiene préstamo activo.
- No se puede prestar a un cliente inactivo.
- No se puede prestar un ejemplar no disponible.
- No se puede devolver dos veces el mismo préstamo.

- Una devolución anticipada suma exactamente 10 puntos.
 - La búsqueda debe estar paginada con máximo 100 resultados por página.
 - La disponibilidad debe cumplir 95% de consultas en 2 segundos o menos con 100.000 ejemplares y 10.000 clientes.
- Base de datos:
- Diseñar tablas para ejemplares, clientes, préstamos y fotos de animales.
 - DNI de cliente debe ser único.
 - idEjemplar debe ser único.
 - Agregar estrategia para consultas eficientes de disponibilidad.
 - Los detalles de índices deben quedar documentados en plan.md o research.md.

API REST:

- CRUD de ejemplares.
- CRUD de clientes.
- Crear préstamo.
- Registrar devolución.
- Consultar disponibilidad.
- Buscar ejemplares con filtros.
- Consultar informe de préstamos activos.

Frontend:

- Pantalla de búsqueda de ejemplares.
- Pantalla de alta/modificación/baja de ejemplares.
- Pantalla de alta/modificación/baja de clientes.
- Pantalla para registrar préstamo.
- Pantalla para registrar devolución.
- Pantalla de informe de préstamos activos.

Testing:

- Tests unitarios para reglas de dominio.
- Tests de servicios para préstamos y devoluciones.
- Tests de repositorio para disponibilidad.
- Tests de integración para endpoints principales.

Generar:

- plan.md
- research.md
- data-model.md
- contracts/openapi.yaml
- quickstart.md

Mantener trazabilidad con spec.md.

No implementar código.

Tasks

/speckit.tasks

Generar tasks.md para implementar el sistema definido en:

```
specs/002-library-management/spec.md
specs/002-library-management/plan.md
specs/002-library-management/data-model.md
specs/002-library-management/contracts/openapi.yaml
specs/002-library-management/quickstart.md
```

No implementar código todavía.

Quiero que las tareas estén organizadas por fases:

1. Setup del backend
2. Modelo de dominio y enums
3. Migraciones Flyway y persistencia
4. Servicios de aplicación
5. API REST
6. Tests backend
7. Setup frontend

8. Pantallas frontend
9. Integración frontend-backend
10. Validación end-to-end según quickstart.md

Reglas:

- Cada tarea debe tener ID único: T001, T002, T003...
- Cada tarea debe indicar archivos concretos.
- Marcar con [P] las tareas paralelizables.
- Indicar dependencias.
- Separar backend y frontend.
- Priorizar primero dominio y tests de reglas de negocio.
- No crear código.
- No ejecutar implementación.

Analyze

/speckit.analyze

Analizar consistencia entre:

specs/002-library-management/spec.md
specs/002-library-management/plan.md
specs/002-library-management/research.md
specs/002-library-management/data-model.md
specs/002-library-management/contracts/openapi.yaml
specs/002-library-management/quickstart.md
specs/002-library-management/tasks.md

No implementar código.

No crear backend ni frontend.

No modificar archivos todavía.

Quiero que detectes:

- requisitos sin tarea
- tareas sin requisito
- inconsistencias entre plan.md y data-model.md
- inconsistencias entre openapi.yaml y tasks.md
- tareas que mezclan demasiadas responsabilidades
- tareas que deberían dividirse
- tareas que deberían tener dependencia explícita
- tareas de frontend que dependan de endpoints no definidos
- reglas de dominio que no tengan test asociado
- posibles violaciones de separación de responsabilidades

Prestar especial atención a:

1. Que Ejemplar no consulte préstamos activos directamente.
2. Que DisponibilidadService calcule disponibilidad final.
3. Que V6__create_indexes.sql esté presente en plan, data-model y tasks.
4. Que las tareas T001-T075 cubran todo el MVP.
5. Que no haya implementación antes de completar análisis.

Devolver un reporte con:

- OK para pasar a taskstoissues, o
- lista de correcciones necesarias antes de seguir.

Implementación controlada T001

Implementé únicamente la tarea T001 según tasks.md.

Fuente de verdad:

- ../specs/002-library-management/spec.md
- ../specs/002-library-management/plan.md
- ../specs/002-library-management/data-model.md
- ../specs/002-library-management/tasks.md

- Reglas:
- Trabajar solo dentro de backend/.
 - No modificar specs/.
 - No avanzar con otras tareas.
 - Respetar exactamente lo definido en plan.md y tasks.md.

Qué se generó y para qué sirve cada archivo

Archivo	Rol	Qué contiene	Uso didáctico
constitution.md	Reglas del proyecto	Principios obligatorios: spec-first, POO real, capas, tests, performance, UX.	Sirve para explicar que la IA también necesita límites.
spec.md	Qué hay que hacer	Historias de usuario, FR, NFR, criterios de aceptación y edge cases.	Sirve para transformar un enunciado en requisitos testeables.
plan.md	Cómo se va a construir	Stack, arquitectura, estructura de carpetas, restricciones y decisiones.	Sirve para separar diseño técnico de implementación.
research.md	Por qué se decidió eso	Justificación de arquitectura, índices, enums, testing, frontend y decisiones diferidas.	Sirve para discutir trade-offs.
data-model.md	Modelo de dominio y BD	Entidades, atributos, reglas de dominio, migraciones e índices.	Sirve para conectar POO con SQL.
openapi.yaml	Contrato API	Endpoints, request/response y errores esperados.	Sirve para dividir front/back sin improvisar.
quickstart.md	Validación E2E	Escenarios curl y resultados esperados.	Sirve para probar antes de “dar por terminado”.
tasks.md	Plan de ejecución	75 tareas en 10 fases, con dependencias, [P] y checkpoints.	Sirve para convertir diseño en trabajo gestionable.

Organización de repositorios

```
biblioteca-sdd/          # repo meta: specs + issues generales
├─ specs/002-library-management/
├─ .specify/
├─ backend/              # repo separado del backend, ignorado por el meta
└─ frontend/             # repo separado del frontend, ignorado por el meta
```

GitHub:

- MartinBarbieri/biblioteca-sdd
- MartinBarbieri/biblioteca-backend
- MartinBarbieri/biblioteca-frontend

El repo meta ignora backend/ y frontend/ para que cada uno tenga su propio historial. Sin embargo, al abrir todo como workspace, el agente puede leer las specs y trabajar dentro de cada repo.

De tasks.md a GitHub Issues

La conversión automática falló cuando Copilot no tenía herramientas de terminal/MCP. La solución práctica fue usar GitHub CLI con gh issue create. Esto también es un aprendizaje didáctico: el flujo SDD no depende de una herramienta única; lo importante es conservar la trazabilidad.

```
gh auth login
gh issue list
# Script PowerShell: leer tasks.md, crear labels, evitar duplicados y crear un issue por T001...T079.
```

Implementación controlada: T001

Una vez creados los issues, la implementación empieza de a una tarea. T001 creó el proyecto Maven base del backend, sin entidades, controladores ni migraciones.

```
Implementá únicamente T001 según:
specs/002-library-management/tasks.md
specs/002-library-management/plan.md
```

```
Trabaja solo dentro de backend/.
No modifiques specs/.
No avances con T002.
```

Regla de oro para implementación

Una tarea, un diff, un commit. Si la IA intenta avanzar con otra cosa, se frena.

.